Timer Module (`timer.v`) – Reference Documentation

Project: PicoRV32 SoC on Real Digital Boolean board (Spartan-7 XC7S50) **Module:** `timer` | **Base address:** `0x4000_0000` | **Bus:** PicoRV32 native memory interface

---

1. Overview

`timer` is a 32-bit memory-mapped hardware timer. It contains a free-running up-counter and a compare register. When the counter reaches the compare value it restarts from zero, sets a **MATCH** flag and raises a level interrupt request. Software reads the counter, programs the compare value, enables or disables the timer, and clears the flag through three registers.

Typical uses:

- Periodic tick (for example 1 ms scheduler tick or 1 s heartbeat).
- Precise delays and time measurement by reading the raw counter.
- Interrupt-driven timing for PicoRV32 (wired to `irq[3]` in the SoC).

Quick facts

Item	Value
Base address	`0x4000_0000` (top.v decodes `mem_addr[31:28] == 4'h4`)
Register width	32 bits, word aligned
Registers	CNT (0x0), CMP (0x4), CTRL (0x8)
Counter	32-bit, counts up by 1 every `clk` cycle while enabled
Reset values	CNT = 0, CMP = `0x05F5_E100` (100,000,000), EN = 1, flags = 0
Reset polarity	**Active LOW** in the current source (see section 9)
Interrupt	`irq` , level, held high until cleared by software

Item	Value
Bus latency	1 wait state (`ready` is a registered one-cycle pulse)
Flip-flops	about 68 (counter 32, compare 32, enable, match, irq, ready)

---

2. How it works

2.1 Counting and compare

On every rising clock edge, if the timer is enabled (`EN = 1`):

1. If `CMP != 0` **and** `CNT >= CMP` : the counter is reset to 0, `MATCH` is set to 1, and `irq` is set to 1.
2. Otherwise the counter increments by 1.

Important consequences (all confirmed by simulation):

- **The period is `CMP + 1` clock cycles.** The counter runs 0, 1, ..., CMP and wraps on the next edge. With `CMP = 9` the flag sets every 10 cycles.
- **The comparison is `>=` , not `==` .** If software lowers CMP below the current CNT, the match fires on the very next cycle instead of waiting for a 32-bit wrap.
- **`CMP = 0` disables matching.** The counter then free-runs over the full 32-bit range and wraps naturally at 2^32 (about 42.95 s at 100 MHz). This is the mode to use for timestamps and delays.
- **Disabling does not reset the counter.** With `EN = 0` the counter holds its value; setting `EN = 1` again resumes from there.
- **MATCH and IRQ are set and cleared together**, so they always have the same value. They are exposed as two separate bits only for convenience.

2.2 Bus interface

The module uses the same handshake as the other peripherals in `top.v` :

- A transaction is `valid && !ready` .
- On that cycle the module registers `ready <= 1` ; `ready` is high for exactly one cycle and the CPU drops `valid` afterwards.

- Read data is a **combinational** multiplexer on `addr[3:0]`. This is safe because the timer registers have no read side effects.
- A write is any transaction with a non-zero `wstrb`; a read is `wstrb == 0`.

2.3 Address decoding inside the module

The module only looks at `addr[3:0]`. The SoC decodes only `addr[31:28]`, so the register block **repeats (aliases) every 16 bytes** across the whole `0x4xxx_xxxx` region (for example `0x4000_0010` reads the counter). Always use the three documented addresses. Offset `0xC` and any unaligned offset read as 0.

---

3. Port list

Port	Dir	Width	Description
`clk`	in	1	System clock (100 MHz on this board)
`reset`	in	1	Synchronous reset, **active LOW** in the current source
`valid`	in	1	Bus request (gate with the timer select in `top.v`)
`ready`	out	1	Bus acknowledge, one-cycle pulse
`wstrb`	in	4	Write byte strobes; `0000` means read
`addr`	in	32	Address (bits `[3:0]` used)
`wdata`	in	32	Write data
`rdata`	out	32	Read data (valid while `ready` is high)
`irq`	out	1	Level interrupt request

---

4. Register map

Address	Name	Access	Reset value	Summary
`0x4000_0000`	`TIMER_CNT`	RO	`0x0000_0000`	Current counter value

Address	Name	Access	Reset value	Summary
`0x4000_0004`	`TIMER_CMP`	RW	`0x05F5_E100`	Compare (period - 1) value
`0x4000_0008`	`TIMER_CTRL`	RW	`0x0000_0001`	Enable, clear, status

4.1 `TIMER_CNT` – offset 0x0 (read-only)

Bits	Field	Description
31:0	`CNT`	Live counter value

- Writes are ignored (the counter is never writable from software).
- Reading while the timer runs returns the value at the instant the bus captures it; two back-to-back reads differ by the cycles elapsed between them.

4.2 `TIMER_CMP` – offset 0x4 (read/write)

Bits	Field	Description
31:0	`CMP`	Compare value. 0 disables matching (free-run mode)

- Byte-wise writes are supported through `wstrb[3:0]`.
- Reset value `100,000,000` gives a period of 100,000,001 cycles = 1.00000001 s at 100 MHz.
- Changes take effect immediately, including while the timer is running.

4.3 `TIMER_CTRL` – offset 0x8 (read/write)

Write behaviour (only `wstrb[0]` is honoured, so bits 31:8 are ignored):

Bit	Name	On write
0	`EN`	Sets the enable state directly: 1 = run, 0 = stop
1	`CLR`	Write 1 to clear MATCH and IRQ (action only, does not store)

Bit	Name	On write
31:2	reserved	Ignored

Read behaviour:

Bit	Name	Meaning
0	`EN`	Timer enabled
1	`IRQ`	Interrupt request currently asserted
2	`MATCH`	Compare match has occurred (same value as bit 1)
31:3	reserved	Read as 0

> **Warning – every write to CTRL also writes EN.** Writing `0x2` (clear only) **stops the timer**, because bit 0 is 0. To clear the flag and keep running, write **`0x3`** (`EN | CLR`). This was confirmed in simulation: after writing `0x2` the register reads back 0 and the counter stops.

If a clear and a new compare match occur in the same cycle, the clear wins and that single match is lost. This is only possible when software clears at the exact cycle the counter wraps and is not a practical concern.

---

5. Configuration and programming guide

5.1 Choosing CMP

`CMP = (f_clk x T) - 1` for a period `T`. At 100 MHz:

Desired period	CMP (decimal)	CMP (hex)
1 us	99	`0x0000_0063`
10 us	999	`0x0000_03E7`
1 ms	99,999	`0x0001_869F`
10 ms	999,999	`0x000F_423F`

Desired period	CMP (decimal)	CMP (hex)
100 ms	9,999,999	`0x0098_967F`
1 s	99,999,999	`0x05F5_E0FF`
Free-run (wraps at 2^32 cycles = 42.95 s)	0	`0x0000_0000`

The power-on default of `100,000,000` is one cycle (10 ns) longer than exactly 1 s.

5.2 C register definitions

```
#include <stdint.h>

#define TIMER_BASE   0x40000000u
#define TIMER_CNT    (*(volatile uint32_t *)(TIMER_BASE + 0x0))
#define TIMER_CMP    (*(volatile uint32_t *)(TIMER_BASE + 0x4))
#define TIMER_CTRL   (*(volatile uint32_t *)(TIMER_BASE + 0x8))

#define TIMER_EN     (1u << 0)   /* write: run   | read: enabled   */
#define TIMER_CLR    (1u << 1)   /* write: clear | read: IRQ status */
#define TIMER_MATCH  (1u << 2)   /* read only                      */
```

5.3 Periodic tick by polling

```
TIMER_CMP  = 100000 - 1;              /* 1 ms at 100 MHz           */
TIMER_CTRL = TIMER_EN | TIMER_CLR;    /* run, and drop any old flag */

for (;;) {
    while (!(TIMER_CTRL & TIMER_MATCH)) ;   /* wait for match        */
    TIMER_CTRL = TIMER_EN | TIMER_CLR;      /* clear, keep running   */
    on_tick();
}
```

5.4 Free-running timestamp and delay

```
TIMER_CMP  = 0;                 /* matching off: counter wraps at 2^32 */
TIMER_CTRL = TIMER_EN;

static inline void delay_cycles(uint32_t n) {
    uint32_t t0 = TIMER_CNT;
    while ((uint32_t)(TIMER_CNT - t0) < n) ;   /* wrap-safe subtraction */
}
```

5.5 Interrupt-driven operation

In the SoC the timer output is connected to **`irq[3]`** of PicoRV32 (`irq[2:0]` are reserved by the core). Steps:

1. Program `TIMER_CMP` and start the timer with `TIMER_CTRL = TIMER_EN | TIMER_CLR`.
2. Unmask interrupt 3 using PicoRV32's `maskirq` custom instruction (the core resets with all interrupts masked). The timer has **no mask register of its own**; masking is done only in the CPU.
3. The interrupt vector is `PROGADDR_IRQ`, which the SoC sets to `0x1000_0010` (inside App RAM), so the application must place its handler there.
4. **Inside the handler, clear the timer** with `TIMER_CTRL = TIMER_EN | TIMER_CLR`. The `irq` line is a level signal; if it is not cleared before returning, the interrupt fires again immediately.

5.6 One-shot operation

There is no hardware one-shot mode. Emulate it by stopping the timer in the handler: `TIMER_CTRL = TIMER_CLR;` (EN = 0, clear flag).

---

6. Timing summary

Parameter	Value
Bus access latency	1 wait state (valid in cycle N, ready in cycle N+1)
Counter increment	1 per `clk` while `EN = 1`
Match period	`CMP + 1` cycles

Parameter	Value
Maximum period	2^32 cycles (42.95 s at 100 MHz)
Interrupt latency to flag	IRQ is set on the same clock edge as MATCH

---

7. SoC integration notes (`top.v`)

```
timer u_timer (
    .clk(clk), .reset(resetn),                 // active-LOW reset: connect
resetn directly
    .valid(mem_valid & timer_sel), .ready(timer_ready),
    .wstrb(mem_wstrb), .addr(mem_addr), .wdata(mem_wdata),
    .rdata(timer_rdata), .irq(timer_irq)
);
```

- `timer_sel = (mem_addr[31:28] == 4'h4)`.
- `mem_ready` is the OR of the core, timer and TRNG ready signals; `mem_rdata` selects `timer_rdata` when `timer_sel` is true.
- `timer_irq` feeds `irq[3]` of the CPU.

---

8. Verification status

The following behaviours were confirmed in simulation (Icarus Verilog) against the exact `timer.v` provided: reset values; period equal to `CMP + 1` cycles; MATCH/IRQ set together; clear by writing `0x3`; write of `0x2` disabling the timer; counter holding while disabled; free-run when `CMP = 0`; 16-byte address aliasing; CNT writes ignored. The module has not been run on hardware by this documentation effort.

---

9. Known issues and notes

1. **Reset polarity.** The file header and port comment do not state the polarity, but the code uses `if (!reset)`, so the module is in reset when `reset = 0`. Connect it to `resetn` (active-low). Connecting `~resetn` would hold the timer in reset during normal operation.

2. **CTRL write semantics.** Every CTRL write overwrites `EN`. Always include bit 0 in writes unless you intend to stop the timer.
3. **Register aliasing.** The block repeats every 16 bytes across the 256 MB region selected by `addr[31:28]`.
4. **Hardware default.** The timer comes out of reset **enabled** with a 1 s period, so MATCH and IRQ become 1 about 1 s after reset even if software never configured it. This is harmless while the CPU interrupt is masked, but software that polls `MATCH` should write `CLR` first.
5. **No separate interrupt-enable bit.** Masking is handled entirely in the CPU.
6. **Single-cycle events.** A match event is lost if clear coincides with the match cycle (see section 4.3).